

Air Quality Intelligence System

Advanced Database Management Systems — Course Report

Péter Dobosi (MW79ON)

May 19, 2026

1 Team Information

Field	Value
Student	Péter Dobosi
Neptun code	MW79ON
Course	Advanced Database Management Systems
Semester	2025/26 Spring
Repository	github.com/dobosipeter/advanced-...-submission
Live system	https://mw79on-demo.online
API docs	https://api.mw79on-demo.online/docs

2 System Architecture

The system is a real-time air quality monitoring and alerting platform ingesting data from the OpenAQ public API into a PostgreSQL database, processing it through triggers and transactions, and exposing it via a FastAPI REST backend and Streamlit GUI.

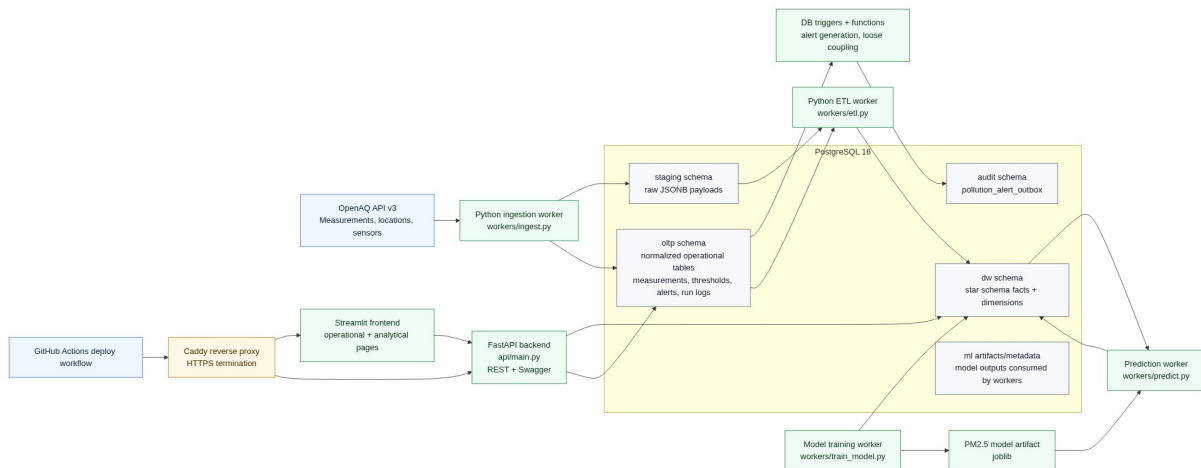


Figure 1: System architecture (boxes-and-arrows) with technology labels.

3 Course Technologies Demonstrated

The following topics from the Advanced DBMS course are actively demonstrated in the project:

3.1 Loose Coupling — Triggers + Log Tables (Chapter 2)

- **Trigger** `trg_measurement_generate_pollution_alert` — fires AFTER INSERT on `oltp.measurement`; matches the measurement value against active threshold rules and inserts an alert row into `oltp.pollution_alert`.
- **Trigger** `trg_pollution_alert_enqueue_event` — fires AFTER INSERT OR UPDATE OF status on `oltp.pollution_alert`; enqueues a lifecycle event into `audit.pollution_alert_outbox` (the log/outbox table).
- This implements the course-prescribed *loose-coupling pattern*: the measurement insert knows nothing about alerting; the alerting trigger knows nothing about downstream consumers. The outbox table decouples producers from consumers.

3.2 Transactions & Error Handling (Chapter 1 review, expanded)

- Ingestion runs inside explicit transactions per location batch.
- Sensor-level **savepoints** allow partial rollback of invalid sensor data without losing valid data from the same location.
- Run status is tracked as **succeeded**, **partial**, or **failed**.
- Invalid measurements (null value, missing timestamp) are skipped and counted rather than rolling back the entire batch.

3.3 Partitioning (Chapter 5)

- `oltp.measurement_raw` is **range-partitioned by month** on `measured_at`.
- 25+ monthly partitions are created via a PL/pgSQL loop (Jan 2025 – Feb 2027).
- Queries filtering by time benefit from **partition pruning**.
- Old partitions can be detached/archived without affecting live operations.

3.4 Database Administration, Backups, Maintenance (Appendix B)

- `scripts/backup.sh`: automated `pg_dump -format=custom` with configurable retention (default 7 days) and optional off-site upload.
- Scheduled via cron: **daily at 02:00 UTC** for full backup.
- VACUUM and ANALYZE are scheduled via `pg_cron` or OS-level cron for table maintenance.

3.5 Indexing (Appendix A)

- B-tree indexes on all foreign key columns and high-frequency query paths.
- Composite indexes: (`sensor_id`, `measured_at`) on `measurement`, (`parameter_id`, `city`, `warning_level`) on `thresholds`.
- Partial index on outbox: `WHERE processed_at IS NULL` for efficient unprocessed-event retrieval.

3.6 Automated Scheduling

The pipeline is scheduled via system cron on the production VM:

- **Every 3 hours**: `run_pipeline.sh ingest-etl` — fetch new data from OpenAQ and load into DW.
- **Daily at 06:00**: `run_pipeline.sh predict-only` — generate next-period predictions.
- **Weekly Sunday 03:00**: `run_pipeline.sh train-predict` — retrain the ML model and predict.
- **Daily at 02:00**: `backup.sh` — database backup with rotation.

4 Functional Specification

1. **Data Ingestion** — Incremental and initial bulk load from OpenAQ v3 API.
2. **Threshold Management** — CRUD for per-city, per-pollutant alert rules (GUI + API).
3. **Automatic Alert Generation** — PostgreSQL trigger on measurement insert.
4. **Outbox/Event Pattern** — Trigger-based loose-coupling for alert lifecycle events.
5. **Alert Review Workflow** — GUI for transitioning alerts (open → reviewed → closed).
6. **Transaction Safety** — Explicit transactions with savepoints and partial-failure handling.
7. **Range Partitioning** — Monthly partitions on measurement table.
8. **Indexing** — B-tree, composite, and partial indexes on hot query paths.
9. **Backup & Maintenance** — Automated pg_dump with retention + scheduled VACUUM.
10. **REST API** — FastAPI with OpenAPI/Swagger auto-documentation.
11. **Operational GUI** — Streamlit pages: Station Explorer, Threshold Management, Alerts, System Status.
12. **CI/CD** — GitHub Actions deploys on push to main via SSH.

5 Logical Data Model (OLTP)

Table	Primary Key	Purpose
ingestion_run_log	ingestion_run_id	Pipeline run tracking
parameter	parameter_id	Pollutant definitions
location	location_id	Monitoring stations
sensor	sensor_id	Physical sensors
measurement_raw	(measurement_id, measured_at)	Observations (partitioned)
threshold_rule	threshold_rule_id	Alert threshold definitions
pollution_alert	pollution_alert_id	Generated alerts

Table 1: OLTP tables in the `oltp` schema.

Supporting schemas:

- `staging.raw_api_response` — raw JSON responses for auditability.
- `audit.pollution_alert_outbox` — event sourcing / loose-coupling log table.

Key Relationships

- `sensor` → `location` (many-to-one), `sensor` → `parameter` (many-to-one)
- `measurement_raw` → `sensor` (FK), → `ingestion_run_log` (FK, nullable)
- `pollution_alert` → `measurement_raw` (composite FK), → `threshold_rule` (FK)

6 Technologies Used

Technology	Role
PostgreSQL 16	RDBMS — OLTP + DW + audit schemas
PL/pgSQL	Trigger functions (alert generation, outbox)
Python 3.12	Worker scripts, API, frontend
FastAPI	REST API with automatic OpenAPI docs
Streamlit	Operational + analytical dashboards
Docker Compose	Container orchestration (5 services)
Caddy 2	Reverse proxy with automatic HTTPS (Let's Encrypt)
GitHub Actions	CI/CD deployment pipeline (SSH + rsync)
Hetzner Cloud	Production VM (Ubuntu 24.04)